Team Notebook

[UNI-FIIS] - SaleEnOdeUno

1 Math

1.1 ntt

```
const ll MOD = 998244353;
vector<vector<int>> w;
int pow2, root;
int smod(int a, int b, int mod) {return (a + b >= mod ? a + b - mod : a + b);}
int rmod(int a, int b, int mod) {return (a - b < 0 ? a - b + mod : a - b);}
int mult(int a, int b, int mod) {return (ll)a * b % mod;}
int qp(int a, int e, int mod) {
    int ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
void calc(int log, int mod) {
    w.resize(log + 1);
    w[0].resize(1, 1);
    for (int l = 1, len = 2; l <= log; l++, len <<= 1) {
        w[l].resize(len / 2);
        int wn = qp(root, pow2 / len, mod);
        for (int j = 0; 2 * j < len; j++) {
            if (j & 1) w[l][j] = mult(w[l - 1][j / 2], wn, mod);
            else w[l][j] = w[l - 1][j / 2];
        }
    }
}
void findroot(int mod) {
    pow2 = 1;
    int k = mod - 1;
    while (!(k & 1)) pow2 <<= 1, k >>= 1;
    root = 1;
    while (qp(root, pow2, mod) != 1 || qp(root, pow2 / 2, mod) == 1) root++;
}
void ntt(vector<int> &a, bool invert, int mod) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <<= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
                int u = a[i + j], v = mult(a[i + j + len / 2], w[l][j], mod);
                a[i + j] = smod(u, v, mod);
                a[i + j + len / 2] = rmod(u, v, mod);
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());
        int invn = qp(n, mod - 2, mod);
        for (int i = 0; i < n; i++) a[i] = mult(a[i], invn, mod);
    }
}
vector<int> polymultmod(vector<int> a, vector<int> b, int mod = MOD) {
    int n = 1, log = 0, rn = a.size() + b.size() - 1;
    while (n < a.size() + b.size() - 1) n <<= 1, log++;
    findroot(mod);
    calc(log, mod);
    a.resize(n), b.resize(n);
    ntt(a, 0, mod), ntt(b, 0, mod);
    for (int i = 0; i < n; i++) a[i] = mult(a[i], b[i], mod);
    ntt(a, 1, mod);
    a.resize(rn);
    return a;
}
```

1.2 fwht bitwiseconvolution

```
//OR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j + h] += a[j];
}
void ifwht(vector<ll> &A) {
    int n = A.size();
    for (int s = n, h = s >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j + h] -= A[j];
}
vector<ll> OrConvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
vector<ll> SubsetConvolution(vector<ll> a, vector<ll> b) {
    int n = 1, bit = 1;
    while (n < a.size() || n < b.size()) n <<= 1, bit++;
    a.resize(n), b.resize(n);
    vector<vector<ll>> A(bit + 1, vector<ll>(n)), B(bit + 1, vector<ll>(n));
    for (int i = 0; i < n; i++) {
        int f = __builtin_popcount(i);
        A[f][i] = a[i], B[f][i] = b[i];
    }
    for (int i = 0; i <= bit; i++) fwht(A[i]), fwht(B[i]);
    vector<ll> conv(n, 0);
    for (int k = 0; k < n; k++) {
        int f = __builtin_popcount(k);
        for (int i = 0; i <= f; i++) conv[k] += A[i][k] * B[f - i][k];
    }
    ifwht(conv);
    return conv;
}
//AND-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j] += a[j + h];
}
void ifwht(vector<ll> &A) {
    int n = A.size();
    for (int s = n, h = n >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j] -= A[j + h];
}
vector<ll> Andconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
//XOR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) {
                ll a1 = a[j + h], a0 = a[j];
                a[j + h] = a0 - a1, a[j] = a0 + a1;
            }
}
void ifwht(vector<ll> &A) {
    fwht(A);
```

```
    int n = A.size();
    for (int i = 0; i < n; i++) A[i] /= n;
}
vector<ll> Xorconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
```

1.3 diophantine

```
ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    x = 1, y = 0;
    ll x1 = 0, y1 = 1;
    while (b) {
        ll q = a / b;
        tie(x, x1) = make_pair(x1, x - q * x1);
        tie(y, y1) = make_pair(y1, y - q * y1);
        tie(a, b) = make_pair(b, a - q * b);
    }
    return a < 0 ? (x *= -1, y *= -1, a *= -1, a) : a;
}
bool diophantine(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (a == 0 && b == 0) return c ? 0 : (x = y = 0, 1);
    g = extended_gcd(a, b, x, y);
    return (c % g == 0 ? (x *= c / g, y *= c / g, 1) : 0);
}
ll divdown(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
}
ll divup(ll a, ll b) {
    return a / b + ((a ^ b) >= 0 && a % b);
}
vector<pll> allsolution(ll a, ll b, ll c, ll lx, ll rx, ll
ly, ll ry) {
    // x = x0 + k*(b/g)    y = y0 - k*(a/g)
    ll x0, y0, g;
    if (!diophantine(a, b, c, x0, y0, g)) return {};
    vector<pll> ans;
    if (!g) {
        for (ll x = lx; x <= rx; x++)
            for (ll y = ly; y <= ry; y++) ans.pb({x, y});
        return ans;
    }
    if (!a) {
        ll y = c / b;
        if (!(ly <= y && y <= ry)) return {};
        for (ll x = lx; x <= rx; x++) ans.pb({x, y});
        return ans;
    }
    if (!b) {
        ll x = c / a;
        if (!(lx <= x && x <= rx)) return {};
        for (ll y = ly; y <= ry; y++) ans.pb({x, y});
        return ans;
    }
    ll dx = b / g, dy = a / g;
    ll kmin, kmax;
    if (dx > 0) kmin = divup(lx - x0, dx), kmax = divdown(rx
- x0, dx);
    else kmin = divup(rx - x0, dx), kmax = divdown(lx - x0,
dx);
    if (dy > 0) kmin = max(kmin, divup(y0 - ry, dy)), kmax =
min(kmax, divdown(y0 - ly, dy));
    else kmin = max(kmin, divup(y0 - ly, dy)), kmax =
min(kmax, divdown(y0 - ry, dy));
    for (ll k = kmin; k <= kmax; k++) ans.pb({x0 + k * dx,
y0 - k * dy});
    return ans;
}
bool positive_sol_minf(ll a, ll b, ll c, ll &x, ll &y, ll
cx, ll cy) {
    // f(x,y)= cx*x + cy*y
    ll g;
    if (!diophantine(a, b, c, x, y, g)) return 0;
    ll dx = b / g, dy = a / g;
    ll kmin = -INF, kmax = INF;
    dx > 0 ? kmin = max(kmin, divup(-x, dx)) : kmax =
min(kmax, divdown(-x, dx));
    dy < 0 ? kmin = max(kmin, divup(y, dy)) : kmax =
min(kmax, divdown(y, dy));
    if (kmin > kmax) return 0;
    ll df = cx * dx - cy * dy;
    ll k = df >= 0 ? kmin : kmax; // df<0 para max
    x += dx * k;
    y -= dy * k;
    return 1;
}
```

1.4 xor basis

```
template <typename T, int LOG>
struct Basis {
    T basis[LOG] = {0};
    int dim = 0;
    bool insert(T x) { // Formará parte del Basis?
        for (int i = LOG - 1; i >= 0; i--)
            if ((x >> i) & 1) {
                if (basis[i]) x ^= basis[i];
                else {
                    basis[i] = x, dim++;
                    return true;
                }
            }
        return false;
    }
    T size() {
        return ((T)1 << dim);
    }
    bool find(T x) { // ¿x ∈ span{x1,x2...}?
        for (int i = LOG - 1; i >= 0; i--) x = min(x, x ^
basis[i]);
        return x == 0;
    }
    T max_xor(T x = 0) { // max (x^a / a ∈ span{x1,x2...})
        for (int i = LOG - 1; i >= 0; i--) x = max(x, x ^
basis[i]);
        return x;
    }
    T kth(T k) { // The k_smaller of span{x1,x2...}
        if (k < 1 || k > ((T)1 << dim)) return -1;
        T x = 0, cont = ((T)1 << dim);
        for (int i = LOG - 1; i >= 0; i--)
            if (basis[i]) {
                if (k > (cont >> 1)) {
                    if (!((x >> i) & 1)) x ^= basis[i];
                    k -= (cont >> 1);
                } else if ((x >> i) & 1) x ^= basis[i];
                cont >>= 1;
            }
        return x;
    }
}
```

1.5 fft

```
typedef complex<double> cd;
vector<vector<cd>> w;
void calc(int log) {
    w.resize(log + 1);
    w[0].resize(1, 1);
    for (int l = 1, len = 2; l <= log; l++, len <<= 1) {
        w[l].resize(len / 2);
        cd wn(cos(2 * PI / len), sin(2 * PI / len));
        for (int j = 0; 2 * j < len; j++)
            w[l][j] = (j & 1 ? w[l - 1][j / 2] * wn : w[l -
1][j / 2]);
    }
}
void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <<= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
```

```
                cd u = a[i + j], v = a[i + j + len / 2] *
w[l][j];
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());
        for (int i = 0; i < n; i++) a[i] /= n;
    }
}
vector<ll> polymult(vector<ll> &a, vector<ll> &b) {
    int n = 1, log = 0;
    while (n < a.size() + b.size() - 1) n <<= 1, log++;
    calc(log);
    vector<cd> A(n);
    for (int i = 0; i < n; i++) A[i] = cd(i < a.size() ?
a[i] : 0, i < b.size() ? b[i] : 0);
    fft(A, 0);
    for (int i = 0; i < n; i++) A[i] *= A[i];
    fft(A, 1);
    vector<ll> ans(a.size() + b.size() - 1);
    for (int i = 0; i < ans.size(); i++) ans[i] =
round(A[i].imag() / 2);
    return ans;
}
// 2D-FFT
void fft2D(vector<vector<cd>> &a, bool invert) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) fft(a[i], invert);
    vector<cd> b(n);
    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) b[i] = a[i][j];
        fft(b, invert);
        for (int i = 0; i < n; i++) a[i][j] = b[i];
    }
}
vector<vector<ll>> convolution2D(vector<vector<ll>> &a,
vector<vector<ll>> &b) {
    int n = 1, logn = 0;
    while (n < a.size() + b.size() - 1) n <<= 1, logn++;
    int m = 1, logm = 0;
    while (m < a[0].size() + b[0].size() - 1) m <<= 1, logm+
+;
    calc(max(logn, logm));
    vector<vector<cd>> A(n, vector<cd>(m));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            A[i][j] = cd(i < a.size() && j < a[0].size() ?
a[i][j] : 0,
                         i < b.size() && j < b[0].size() ?
b[i][j] : 0);
    fft2D(A, 0);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) A[i][j] *= A[i][j];
    fft2D(A, 1);
    vector<vector<ll>> ans(a.size() + b.size() - 1,
vector<ll>(a[0].size() + b[0].size() - 1));
    for (int i = 0; i < ans.size(); i++)
        for (int j = 0; j < ans[0].size(); j++) ans[i][j] =
round(A[i][j].imag() / 2);
    return ans;
}
```

1.6 millerrabin polardrho

```
ll mult(ll a, ll b, ll mod) {
    return (int128)a * b % mod;
}
ll qp(ll a, ll e, ll mod) {
    ll ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
bool compositetest(ll a, ll d, ll s, ll n) {
    ll x = qp(a, d, n);
    if (x == 1 || x == n - 1) return 0;
    for (ll i = 0; i < s - 1; i++) {
        x = mult(x, x, n);
        if (x == n - 1) return 0;
    }
    return 1;
}
bool esprimo(ll n) {
    if (n < 2) return 0;
    ll d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;
    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
37}) {
        if (n == a) return 1;
        if (compositetest(a, d, s, n)) return 0;
    }
    return 1;
}
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
ll random(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng);
}
ll f(ll x, ll c, ll mod) {
    ll m = mult(x, x, mod);
    return m + c >= mod ? m + c - mod : m + c;
}
ll getfactor(ll n) {
    if (n % 2 == 0) return 2;
    if (esprimo(n)) return n;
    while (true) {
        ll y = random(1, n - 1);
        ll c = random(1, n - 1);
        ll m = 64, g = 1, r = 1, q = 1;
        ll x, ys;
        while (g == 1) {
            x = y;
            for (ll i = 0; i < r; i++) y = f(y, c, n);
            ll k = 0;
            while (k < r && g == 1) {
                ys = y;
                ll lim = min(m, r - k);
                for (ll i = 0; i < lim; i++) {
                    y = f(y, c, n);
                    ll diff = x > y ? x - y : y - x;
                    q = mult(q, diff, n);
                }
                g = __gcd(q, n);
                k += m;
            }
            r <<= 1;
        }
        if (g == n) {
            do {
                ys = f(ys, c, n);
                ll diff = x > ys ? x - ys : ys - x;
                g = __gcd(diff, n);
            } while (g == 1);
        }

        if (g > 1 && g < n) return g;
    }
}
vector<pll> fp(ll n) {
    stack<ll> st;
    st.push(n);
    vector<pll> ans;
    map<ll, ll> cont;
    while (!st.empty()) {
        ll x = st.top();
        st.pop();
        if (x == 1) continue;
        if (esprimo(x)) cont[x]++;
        else {
            ll f = getfactor(x);
            st.push(f), st.push(x / f);
        }
    }
    for (auto [x, f] : cont) ans.pb({x, f});
    return ans;
}
```

2 Graphs

2.1 dinic

```
// This implementation is for small graphs G(V, E) (|V| <=
3000 aprox)
// where O(|V|^2) memory will fit the memory limit.
using Cap = long long;
const int MX = 3000;
const Cap INF = 1e18;
struct Graph {
    vector<int> adj[MX];
    Cap cap[MX][MX];
    Cap flow[MX][MX];
    bool added[MX][MX];
    int level[MX];
    int nextEdge[MX];

    void clear(int n) {
        for (int i = 0; i < n; i++) {
            adj[i].clear();
            for (int j = 0; j < n; j++) cap[i][j] = 0,
flow[i][j] = 0, added[i][j] = false;
        }
    }

    void addEdge(int u, int v, Cap w, bool dir) {
        if (!added[u][v]) {
            adj[u].push_back(v);
            adj[v].push_back(u);
            added[u][v] = added[v][u] = true;
        }
        cap[u][v] += w;
        if (!dir) cap[v][u] += w;
    }

    Cap dfs(int u, int t, Cap f) { // O(E)
        if (u == t) return f;
        for (int &i = nextEdge[u]; i < adj[u].size(); i++) {
            int v = adj[u][i];
            Cap cf = cap[u][v] - flow[u][v];
            if (cf == 0 || level[v] != level[u] + 1)
continue;
            Cap ret = dfs(v, t, min(f, cf));
            if (ret > 0) {
                flow[u][v] += ret;
                flow[v][u] -= ret;
                return ret;
            }
        }
        return 0;
    }

    bool bfs(int s, int t, int n) { // O(E)
        fill(level, level + n, -1);
        queue<int> q({s});
        level[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            nextEdge[u] = 0;
            q.pop();
            for (int v : adj[u]) {
                Cap cf = cap[u][v] - flow[u][v];
                if (level[v] == -1 && cf > 0) {
                    level[v] = level[u] + 1;
                    q.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    Cap maxFlow(int s, int t, int n) {
        // General: O(E * V^2), O(E * V + V * |F|)
        // Unit Cap: O(E * min(E^(1/2) , V^(2/3)))
        // Unit Network: O(E * V^(1/2))
        // In unit network, all the edges have unit capacity
        // and for any vertex except s and t either the
        // incoming or outgoing edge is unique.
        Cap ans = 0;
        while (bfs(s, t, n)) { // O(V) iterations
            // after bfs, the graph is a DAG
            while (Cap inc = dfs(s, t, INF)) ans += inc;
        }
        return ans;
    }
} G;
```

3 Data Structure

3.1 ordered set

```
// find_by_order(k) -> return iterator to the k-th element
(0-indexed) - O(log n)
// order_of_key(num) -> # of items strictly smaller than num
- O(log n)
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

ordered_set<int> S;
```

3.2 sparse table

```
const int LG = __lg(MX) + 1;
int st[MX][LG];
void build(vector<int> &A) { // O(|f| n log n)
    int n = A.size();
    for (int i = 0; i < n; i++) st[i][0] = A[i];
    for (int k = 1; k < LG; k++) {
        for (int i = 0; i + (1 << k) <= n; i++) {
            st[i][k] = min(st[i][k - 1], st[i + (1 << (k -
1))][k - 1]);
        }
    }
}
int query(int l, int r) { // O(|f|)
    int lg = __lg(r - l + 1);
    return min(st[l][lg], st[r - (1 << lg) + 1][lg]);
}
```

3.3 pst lazy

```
const int MX = 2e5 + 5;
const int Q = 2e5 + 5;
const int LOG = 20;
const int ND = 2 * MX + 6 * Q * LOG;
const ll NEUT = 0;
const ll NEUT_LAZY = -2 * INF;
struct nod {
    ll t, lazy;
    nod *l, *r;
};
nod nods[ND];
int id = 0;
nod *crearnodo(nod *u = NULL) {
    if (u) nods[id].t = u->t, nods[id].lazy = u->lazy,
nods[id].l = u->l, nods[id].r = u->r;
    else nods[id].t = NEUT, nods[id].lazy = NEUT_LAZY,
nods[id].l = nods[id].r = NULL;
    return &nods[id++];
}
ll f(ll a, ll b) {
    return a + b;
}
void apply(ll val, nod *u, ll l, ll r) {
    u->t += (r - l + 1) * val;
    if (u->lazy == NEUT_LAZY) u->lazy = val;
    else u->lazy += val;
}
struct PST {
    nod *root[MX + 5];
    ll n;
    ll lastversion = -1;
    PST(vector<ll> &a) : n(a.size()) {
        root[++lastversion] = crearnodo();
        build(a, root[0], 0, n - 1);
    }
    void build(vector<ll> &a, nod *&u, ll l, ll r) {
        if (!u) u = crearnodo();
        if (l == r) u->t = a[l];
        else {
```

```
            ll m = (l + r) >> 1;
            build(a, u->l, l, m);
            build(a, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
    }
    void push(nod *u, ll l, ll r) {
        if (u->lazy == NEUT_LAZY || l == r) return;
        ll m = (l + r) >> 1;
        u->l = crearnodo(u->l), u->r = crearnodo(u->r);
        apply(u->lazy, u->l, l, m);
        apply(u->lazy, u->r, m + 1, r);
        u->lazy = NEUT_LAZY;
    }
    nod *update(ll ql, ll qr, ll val, nod *prev, ll l, ll r)
{
        if (qr < l || r < ql) return prev;
        nod *u = crearnodo(prev);
        if (ql <= l && r <= qr) apply(val, u, l, r);
        else {
            push(u, l, r);
            ll m = (l + r) >> 1;
            u->l = update(ql, qr, val, u->l, l, m);
            u->r = update(ql, qr, val, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
        return u;
    }
    void update(ll ql, ll qr, ll val, ll version = -1) {
        if (version == -1) version = lastversion;
        root[++lastversion] = update(ql, qr, val,
root[version], 0, n - 1);
    }
    ll query(ll ql, ll qr, nod *u, ll l, ll r) {
        if (!u || qr < l || r < ql) return NEUT;
        if (ql <= l && r <= qr) return u->t;
        push(u, l, r);
        ll m = (l + r) >> 1;
        return f(query(ql, qr, u->l, l, m), query(ql, qr, u-
>r, m + 1, r));
    }
    ll query(ll ql, ll qr, ll version = -1) {
        if (version == -1) version = lastversion;
        return query(ql, qr, root[version], 0, n - 1);
    }
};
```

3.4 multi ordered set

```
// find_by_order(k) -> return iterator to the k-th element
(0-indexed) - O(log n)
// order_of_key(num) -> # of items strictly smaller than num
- O(log n)

// using less_equal cause that lower_bound works as
upper_bound and viceversa
// also .find() does not work and you can only erase by
iterator now
// using for example erase(upper_bound(x))
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;

ordered_set<int> s;
```